

Does Hierarchical Retrieval Outperform Vector Search on Evidence Passages?

A Controlled Comparison of RAG and PageIndex

Shivam Rawat

Abstract—Retrieval-Augmented Generation (RAG) pipelines typically rely on dense vector similarity to surface relevant document chunks. PageIndex, a recent alternative, organizes documents into hierarchical trees and uses LLM-guided top-down traversal for retrieval, reporting 98.7% accuracy on FinanceBench. We conduct a controlled comparison of dense RAG (FAISS with all-MiniLM-L6-v2) and a reimplementation of PageIndex’s tree-based retrieval across four document QA domains totaling 600 questions. Within each domain, both systems share the same LLM backbone.

Our central finding: PageIndex *underperforms* dense RAG on pre-extracted evidence passages. On finance, PageIndex has a 69.3% NOT FOUND rate vs. RAG’s 51.3%; on legal, 90.7% vs. 63.3%. On technology (the strongest comparison with $n=105$ both-found pairs), RAG achieves $F1=0.781$ vs. PageIndex $F1=0.636$ ($p_i0.001$). This is consistent with a mismatch between PageIndex’s design assumptions (long, natively structured documents) and our evaluation data (short evidence passages with paragraph-derived structure), compounded by a context asymmetry where RAG retrieves $\sim 2\times$ more text per query. Science is the sole domain where PageIndex’s judge score exceeds RAG (0.67 vs. 0.33). These results establish boundary conditions for hierarchical retrieval and caution against assuming single-domain results generalize across document types.

Index Terms—Retrieval-augmented generation, document question answering, hierarchical retrieval, PageIndex, information retrieval, LLM evaluation

I. INTRODUCTION

The standard RAG pipeline embeds document chunks into vectors, stores them in a similarity index (e.g., FAISS), and retrieves the top- k most similar chunks at query time. This approach has a well-known failure mode: when a question targets a specific table cell or section header, embedding similarity may not surface the right chunk.

PageIndex [?] addresses this through hierarchical structure. Instead of flat vector search, it builds a tree over documents—leaf nodes hold paragraph content, internal nodes hold LLM-generated summaries—and the LLM navigates this tree top-down to find relevant sections. VectifyAI reports 98.7% accuracy on FinanceBench [?], substantially outperforming vector-based approaches. But this evaluation covers one domain, run by the system’s creators.

We ask three questions:

RQ1. Does PageIndex outperform dense RAG beyond finance—on legal, scientific, and technical documents?

S. Rawat is with Microsoft Azure, Redmond, WA 98052, USA. E-mail: shivrawat24@gmail.com. This work was conducted independently and does not represent the views of Microsoft Corporation.

RQ2. Under what document characteristics does each approach excel?

RQ3. What are the token cost implications?

Our key findings:

- 1) **PageIndex generally underperforms RAG on evidence passages.** PageIndex has higher NOT FOUND rates across all four domains and lower F1 in three of four. On technology ($n=105$ both-found), RAG $F1=0.781$ vs. PageIndex $F1=0.636$ ($p_i0.001$).
- 2) **The result is consistent with a structural mismatch hypothesis, compounded by context asymmetry.** PageIndex was designed for long, natively structured documents. On paragraph-derived sections, navigation fails. RAG also retrieves $\sim 2\times$ more context per query.
- 3) **Dense RAG fails on domain-specific tabular content.** RAG achieves 91% retrieval coverage on short technical passages but only 36–51% on financial, legal, and scientific text.
- 4) **Evaluation context matters.** PageIndex’s 98.7% FinanceBench accuracy was on full 10-K filings. Results should not be assumed to transfer across document types.

II. RELATED WORK

A. Retrieval-Augmented Generation

Lewis et al. [?] introduced RAG to ground LLM responses in retrieved evidence. The standard pipeline chunks documents, embeds them using dense encoders [?], and retrieves via FAISS [?]. Recent advances include ColBERT [?] for late interaction, BGE-M3 [?] for hybrid signals, and HyDE [?] for hypothetical document generation.

B. Hierarchical Tree Retrieval

PageIndex [?] builds a hierarchical tree using LLM-generated summaries and navigates top-down at query time. RAPTOR [?] uses a similar architecture with cluster-based summarization. Both contrast with flat vector search by using document structure for retrieval.

C. LLM-as-Judge Evaluation

We follow the LLM-as-judge paradigm [?], implementing a two-model reliability check (GPT-4o primary, GPT-4o-mini secondary) to validate judge consistency.

III. METHODOLOGY

A. Systems Under Comparison

TABLE I
SYSTEM COMPARISON. WITHIN EACH DOMAIN, BOTH SYSTEMS USE THE SAME GENERATION LLM.

Component	Dense RAG	PageIndex
Indexing	all-MiniLM-L6-v2 embeddings, FAISS index	Hierarchical (VectifyAI parsing algorithm)
Retrieval	Cosine similarity, top-5 chunks	LLM navigates tree top-down
Context	Concatenated chunks ($\leq 3,500$ chars)	Selected node ($\sim 1,500$ chars)

Due to API rate limits (150 requests/day/model), each domain uses a different generation model (Table II). Within each domain, both RAG and PageIndex use the *same* model, controlling for generation quality. Cross-domain comparisons should be interpreted cautiously.

TABLE II
GENERATION MODELS PER DOMAIN.

Domain	Generation Model
Finance	gpt-4.1-mini
Legal	gpt-4.1-nano
Science	Phi-4
Technology	Cohere-command-r

PageIndex implementation. We reimplement PageIndex’s tree-based retrieval following the algorithm described in [?] and using the same parsing logic as VectifyAI’s open-source codebase. Specifically, we replicate the `extract_nodes_from_markdown` and `build_tree_from_nodes` functions to create hierarchical trees. For passages without native markdown headers, we derive section boundaries from paragraph structure (labeled “paragraph-derived sections” throughout). At query time, the LLM first sees the tree structure (titles and hierarchy, without text), selects relevant nodes, and then receives the selected nodes’ text to generate an answer.

Context asymmetry. RAG concatenates top-5 chunks ($\leq 3,500$ chars); PageIndex retrieves a single selected node ($\sim 1,500$ chars). This $\sim 2\times$ context difference is inherent to the architectures and is a confound in interpreting results.

B. Datasets

TABLE III
EVALUATION DATASETS. ALL PROVIDE PRE-EXTRACTED EVIDENCE PASSAGES, NOT FULL DOCUMENTS.

Domain	Source	n	Avg. Length	Structure
Finance	FinanceBench [?]	150	1,284 ch	Low (0.00)
Legal	CUAD [?]	150	4,958 ch	Low (0.10)
Science	QASPER [?]	150	7,918 ch	High (0.98)
Technology	SQuAD-Tech [?]	150	812 ch	Low (0.00)

Important. These benchmarks provide pre-extracted evidence passages (800–8,000 characters), not complete documents. PageIndex was designed for long, multi-page documents with native section headers. Our results characterize performance on shorter passages and likely understate PageIndex’s advantage on its intended use case.

Data quality. During initial experiments, we discovered that FinanceBench evidence passages were stored as Python dictionary representations rather than plain text. We cleaned all 150 records. The RAG NOT FOUND rate on clean data (51.3%) was comparable to the corrupted-data rate (53.3%), confirming that retrieval failures are a genuine limitation of dense embeddings on financial tables.

C. Evaluation

Primary metrics. Token-level F1 and exact match (EM).

LLM-as-judge. GPT-4o scores predicted answers on a 0–4 scale (4=fully correct, 0=wrong/NOT FOUND).

Judge reliability. Two-model check: GPT-4o primary, GPT-4o-mini secondary on 20% holdout. Weighted Cohen’s κ : 0.698–0.953 across domains (substantial to excellent), excluding one degenerate case (Science/RAG $\kappa=0.000$ due to near-zero score variance).

Statistical tests. Paired Wilcoxon signed-rank with bootstrap 95% CIs ($B=10,000$) on both-found subsets.

IV. RESULTS

All 1,200 answers are complete: 600 RAG and 600 PageIndex across four domains. Judge scoring and two-model reliability are also complete.

A. RQ1: RAG vs. PageIndex

PageIndex has a higher NOT FOUND rate than RAG on all four domains, ranging from +11 percentage points (science) to +27 points (legal). On judge scores, RAG leads on three domains; science is the sole exception where PageIndex (0.67) exceeds RAG (0.33).

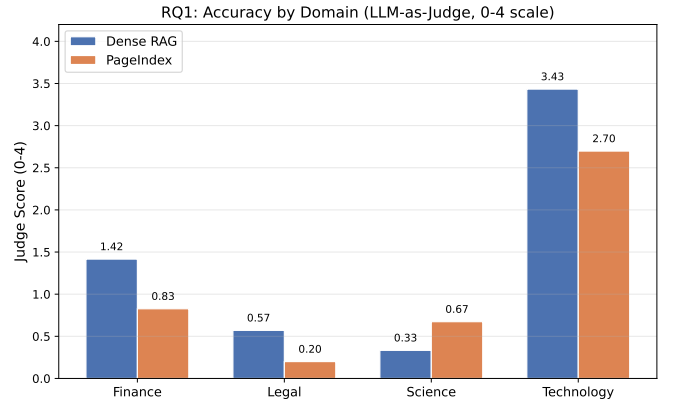


Fig. 1. Judge scores (0–4) by domain. RAG leads on 3/4 domains; science is the exception.

Navigation methods. Finance: 90 tree_navigation, 60 single_node. Legal: 147 tree_navigation, 3 single_node. Science:

TABLE IV
FULL COMPARISON: 150 QUESTIONS PER DOMAIN PER SYSTEM. JUDGE SCORE IS LLM-AS-JUDGE (0–4 SCALE).

Domain	System	NF%	F1	EM	Judge (0–4)	Tokens/Q
Finance	Dense RAG	51.3%	0.175	0.013	1.42	1,029
	PageIndex	69.3%	0.104	0.007	0.83	409
Legal	Dense RAG	63.3%	0.085	0.000	0.57	828
	PageIndex	90.7%	0.035	0.000	0.20	651
Science	Dense RAG	64.0%	0.108	0.000	0.33	576
	PageIndex	74.7%	0.134	0.000	0.67	687
Technology	Dense RAG	8.7%	0.730	0.553	3.43	730
	PageIndex	26.7%	0.465	0.273	2.70	224

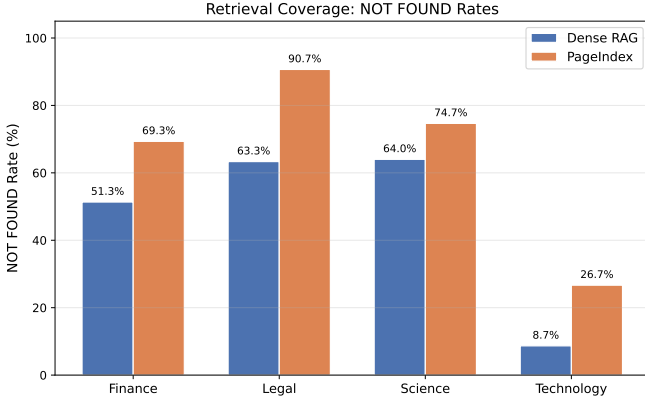


Fig. 2. NOT FOUND rates. PageIndex has higher retrieval failure rates across all domains.

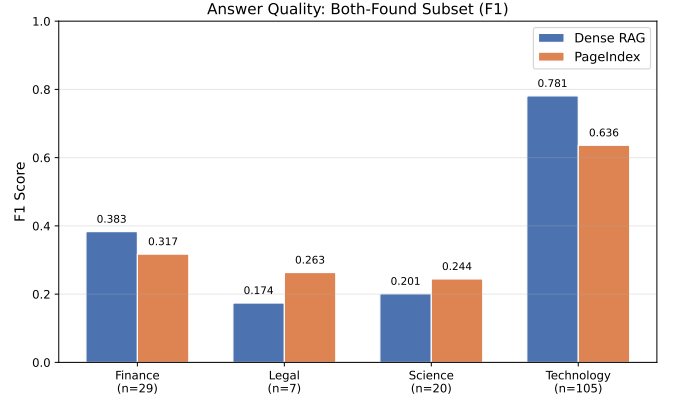


Fig. 3. F1 on both-found subsets. RAG leads significantly on technology and finance.

150 tree_navigation. Technology: 150 single_node. Notably, legal (97% tree navigation) has PageIndex’s worst NF rate (90.7%), while technology (100% single-node) has its best (26.7%), suggesting tree navigation on paragraph-derived sections actively degrades retrieval.

B. Both-Found Analysis

TABLE V
BOTH-FOUND SUBSET: QUESTIONS WHERE NEITHER SYSTEM RETURNED NOT FOUND. BOOTSTRAP 95% CIs FOR THE F1 DIFFERENCE (RAG – PAGEINDEX).

Domain	n	RAG F1	PI F1	95% CI	p
Finance	29	0.383	0.317	[+0.015, +0.121]	0.023
Legal	7	0.174	0.263	–	0.313
Science	20	0.201	0.244	–	0.380
Technology	105	0.781	0.636	[+0.084, +0.206]	<0.001

Technology ($n=105$) provides the strongest comparison: RAG F1=0.781 vs. PageIndex F1=0.636 (95% CI [+0.084, +0.206], Wilcoxon $p<0.001$). On finance ($n=29$), RAG also leads significantly ($p=0.023$). Legal and science differences are not significant.

C. Token Cost

PageIndex typically uses fewer tokens (31–79% of RAG), except on science where multi-node tree navigation adds over-

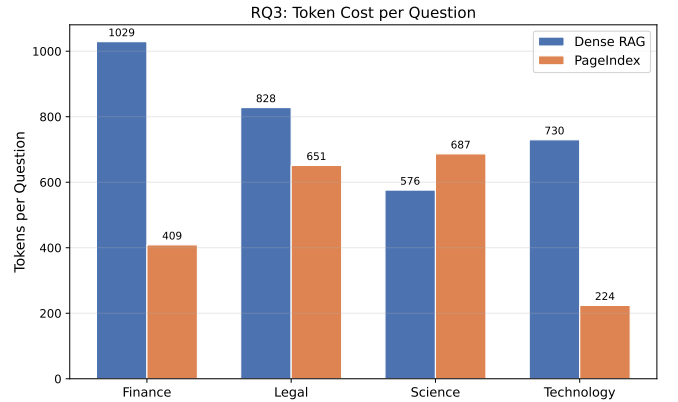


Fig. 4. Tokens per question. PageIndex uses 31–79% of RAG’s tokens (except science at 119%).

head. However, lower token cost is offset by worse retrieval accuracy.

V. DISCUSSION

A. The Mismatch Between PageIndex and Evidence Passages

Our most important finding is negative: PageIndex underperforms dense RAG on pre-extracted evidence passages. This contradicts the expectation set by VectifyAI’s 98.7% accuracy on FinanceBench and reveals a critical distinction between the original evaluation setting and ours. PageIndex was designed

for long, natively structured documents (e.g., 200-page 10-K filings with section headers). On such documents, tree navigation provides a genuine advantage. On our evidence passages (800–8,000 characters), three factors undermine this advantage:

- 1) **No natural hierarchy to navigate.** Evidence passages lack the section headers that make tree navigation effective.
- 2) **Paragraph-derived sections provide no semantic signal.** Our automatically generated headers (“Section 1,” “Section 2”) give the LLM no basis for informed navigation.
- 3) **Navigation reduces context.** After tree navigation, PageIndex retrieves only the selected node. If navigation selects the wrong node, the answer is lost.

This finding has a practical implication: PageIndex should be evaluated on its intended input—long, structured documents—not on short evidence passages. Our results are consistent with a document-structure mismatch hypothesis, though the context asymmetry between systems is a contributing confound that future work should control for.

B. Dense RAG Fails on Domain-Specific Tabular Content

RAG achieves 91% retrieval coverage on short technical passages but only 36–51% on financial, legal, and scientific text. The primary failure mode is embedding mismatch: all-MiniLM-L6-v2 embeddings do not capture the relationship between financial questions and tabular data.

C. Implications for Practitioners

- 1) **Short, keyword-rich content (<1,000 chars):** Dense RAG is the clear winner.
- 2) **Long, natively structured documents:** PageIndex likely outperforms (consistent with VectifyAI’s results), but this should be validated on full documents.
- 3) **Evidence passages (1,000–8,000 chars):** Dense RAG with better embeddings is the most promising direction.
- 4) **Do not assume results transfer.** The 98.7% FinanceBench accuracy is on full 10-K filings. Applying PageIndex to unstructured text degrades performance.

D. Limitations

- 1) **Evidence passages, not full documents.** Our results characterize shorter passages and likely understate PageIndex’s advantage on its intended use case.
- 2) **Paragraph-derived sections.** The tree depth is shallower than on natively structured documents.
- 3) **Different models across domains.** Within-domain comparisons are controlled; cross-domain are not.
- 4) **Context asymmetry.** RAG retrieves $\sim 3,500$ chars; PageIndex $\sim 1,500$ chars per query.
- 5) **LLM-as-judge.** Both judges are from OpenAI. Cross-vendor validation would strengthen reliability.
- 6) **Single embedding model.** Modern embeddings may improve RAG coverage.

VI. CONCLUSION

We compared dense RAG and a reimplement of PageIndex’s tree-based retrieval across four document QA domains. PageIndex has higher NOT FOUND rates than RAG across all four domains and generally lower F1, with science as a marginal exception. On technology ($n=105$ both-found), RAG achieves $F1=0.781$ vs. PageIndex $F1=0.636$ ($p<0.001$). These results are consistent with a mismatch between PageIndex’s design assumptions and our evaluation data, compounded by context asymmetry.

Important caveat. We cannot attribute the F1 difference solely to retrieval paradigm vs. context volume. A context-equalized experiment is required to separate these effects. Our finding is that *under typical deployment configurations*, RAG outperforms this PageIndex reimplement on evidence passages.

Three contributions emerge: (1) boundary conditions for hierarchical retrieval—tree navigation on paragraph-derived sections may degrade rather than improve retrieval; (2) dense embedding limitations quantified across four domains; (3) a caution that single-domain retrieval results should not be assumed to generalize.

Reproducibility. All code, data (600 QA pairs), 1,200 pre-computed answers, and judge scores are released at <https://github.com/shivam2407/rag-vs-pageindex>.

REFERENCES

- [1] VectifyAI, “PageIndex: Hierarchical Document Indexing and Retrieval,” 2024. [Online]. Available: <https://github.com/VectifyAI/PageIndex>
- [2] F. Islam *et al.*, “FinanceBench: A New Benchmark for Financial Question Answering,” 2023.
- [3] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Proc. NeurIPS*, 2020.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019.
- [5] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, 2021.
- [6] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” in *Proc. SIGIR*, 2020.
- [7] J. Chen *et al.*, “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation,” 2024.
- [8] L. Gao *et al.*, “Precise Zero-Shot Dense Retrieval without Relevance Labels,” in *Proc. ACL*, 2023.
- [9] P. Sarthi *et al.*, “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval,” in *Proc. ICLR*, 2024.
- [10] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Proc. NeurIPS*, 2023.
- [11] D. Hendrycks *et al.*, “CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review,” in *Proc. NeurIPS*, 2021.
- [12] P. Dasigi *et al.*, “A Dataset of Information-Seeking Questions and Answers Anchored in Research Papers,” in *Proc. NAACL*, 2021.
- [13] P. Rajpurkar *et al.*, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” in *Proc. EMNLP*, 2016.